Dense Semantic Retrieval Architectures for Document Question Answering
Authors: MCA Research Laboratory

ABSTRACT
Retrieval-Augmented Generation (RAG) integrates dense passage retrieval with pre-trained sequence-to-sequence la

1. INTRODUCTION & PROBLEM FORMULATION
Standard large language models (LLMs) struggle with factual hallucinations and lack verifiable provenance when answ

2. DOCUMENT INGESTION & HIGH-THROUGHPUT EXTRACTION

The document ingestion pipeline implements high-throughput PDF parsing coupled with rigorous text sanitation before

2.1 High-Throughput PDF Extraction

PDF extraction is performed using PyMuPDF (Python bindings for the MuPDF C-library), achieving a 15x parsing thro

2.2 Page Provenance & Font Handling

During extraction, text blocks are tagged with exact bounding box coordinates and font size metadata, enabling accura

3. SANITATION & TEXT REGULARIZATION

Extracted text is preprocessed through a multi-pass normalization pipeline:

3.1 Whitespace Regularization

Multiple consecutive spaces, non-breaking spaces, horizontal tabs, and inconsistent line endings are converted to sta

3.2 Hyphenation Resolution

Trailing hyphens at line breaks (hyphen-wrap) caused by column justification are recombined into single words.

3.3 Control Code Filtering & Unicode Normalization

Non-printable ASCII control characters (0x00 to 0x1F) are stripped, and text is converted to Unicode NFC canonical d

4. RECURSIVE CHUNKING & SEPARATOR HIERARCHY

Granular passage segmentation is crucial for dense vector retrieval accuracy.

4.1 Target Window & Overlap

We employ a recursive chunking strategy with a target window size of 1000 characters and a sliding overlap of 150 ch

4.2 Separator Priority

Text splitting respects a hierarchical separator priority:
['\n\n', '\n', '. ', '? ', '! ', '; ', ', ', ' ', '']
This hierarchical splitting ensures syntactic cohesion while guaranteeing that sentence boundaries are preserved whe

5. DENSE EMBEDDING GENERATION

5.1 Bi-Encoder Model Architecture
Each chunk is encoded into a 384-dimensional dense semantic vector space using the all-MiniLM-L6-v2 transformer m

5.2 L2 Normalization
Embeddings are normalized using L2 unit norm (||v||_2 = 1.0). This mathematical normalization ensures that the inner

6. VECTOR INDEXING & BASELINE RETRIEVAL PERFORMANCE

Embeddings and chunk metadata are indexed in ChromaDB using Hierarchical Navigable Small World (HNSW) graph

6.1 Latency Benchmarks

Vector similarity search demonstrates an average query latency of 45.2ms when retrieving candidate pools of size K =

6.2 Retrieval Baseline Results

On a test set of technical queries, baseline vector-only retrieval achieves a Hit Rate@5 of 82.0% and a Mean Recipro

7. ABLATION STUDY: CHUNK SIZE AND OVERLAP

We systematically evaluated chunk sizes of 500, 1000, and 1500 characters on retrieval performance:
1. 500-character chunks: High precision but fragmented context, reducing downstream Gemini synthesis quality.
2. 1000-character chunks: Optimal empirical balance, achieving 82.0% Hit Rate@5 while retaining full conceptual sen
3. 1500-character chunks: Caused embedding vector dilution where key facts were overshadowed by peripheral sente
An overlap of 150 characters eliminated boundary information loss across adjacent chunks.

8. LIMITATIONS OF SINGLE-STAGE VECTOR RETRIEVAL

Detailed failure analysis of vector search reveals three primary vulnerability modes:
1. Lexical Specificity Failure: Queries with rare identifiers or version numbers receive lower similarity scores.
2. Negation Blindness: Bi-encoders frequently map 'system X supports Y' and 'system X does not support Y' to near-id
3. Polysemy Confusion: Technical terms with multiple contextual meanings across documents lead to spurious candid